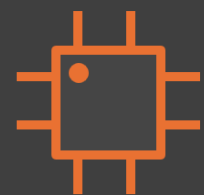


بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Verilog



It's been a long run,

you are here

more.pending



Verilog

RUNNING



STA

QUEUED



Power

QUEUED



CDC

QUEUED



RDC

QUEUED



DFT

QUEUED



Synth

QUEUED



...

But We haven't \$arrived yet.



GitHub

code examples



Drive

full files and slides



LinkedIn

new episode alerts

Introduction

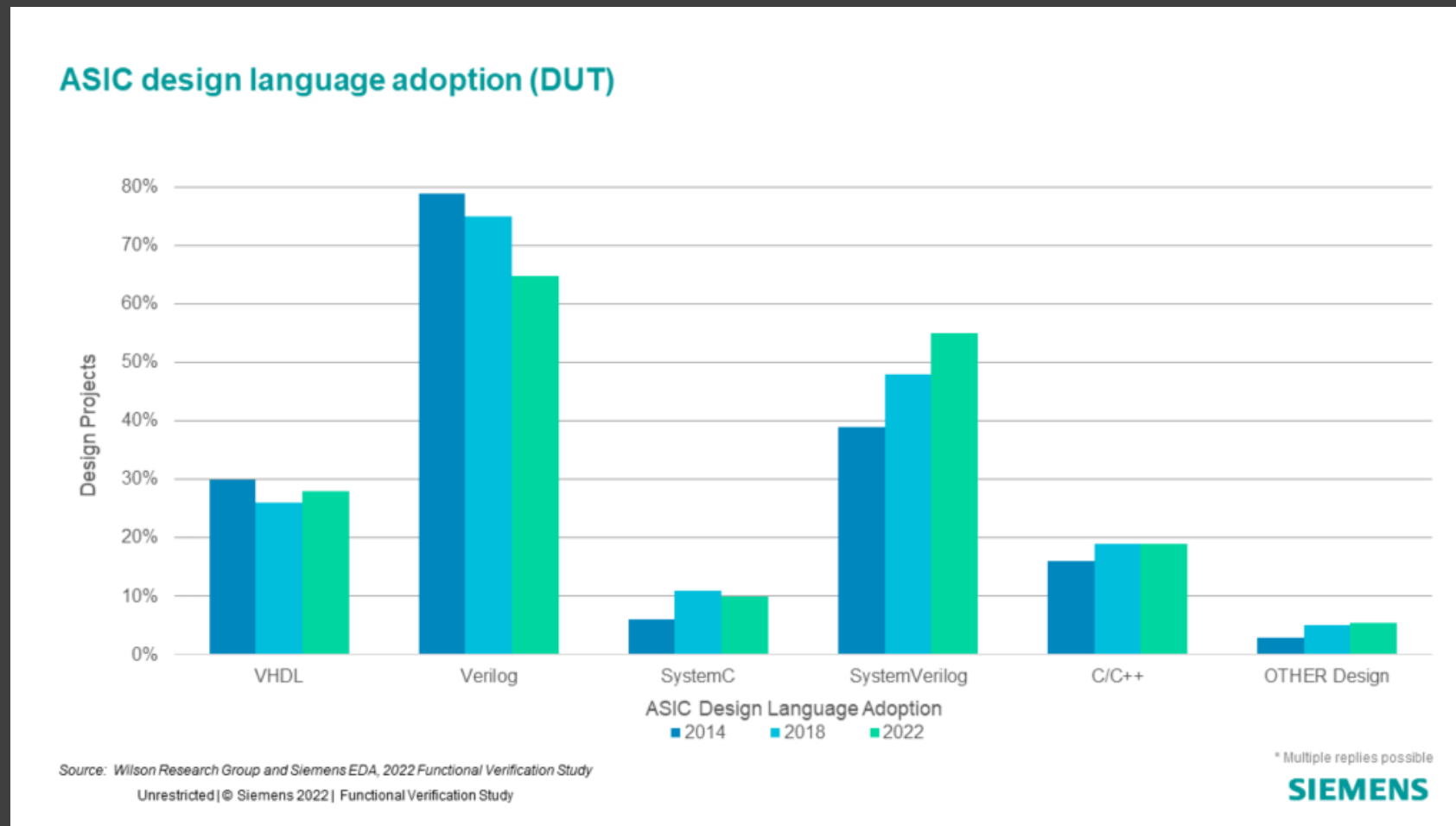
Introduction

What is HDL languages

- hardware description language (HDL) that is used to describe digital systems and circuits in the form of code.
- They are not an algorithm-based language like C or python, and it focus more to translate the Hardware into a code that describes it.
- HDL languages are:
 - VHDL
 - Verilog “our main topic”
 - System Verilog “Later”
- Verilog is widely used for design and verification of digital and mixed-signal systems, including both application-specific integrated circuits (ASICs) and field-programmable gate arrays (FPGAs). It supports a range of levels of abstraction, from structural to behavioral, and is used for both simulation-based design and synthesis-based design.
- There are multiple of ways to describe hardware using Verilog like gate-level, RTL-level, and behavioral-level modeling.

Introduction

This graph shows that Verilog is the most used language in IC design.



Introduction

Verilog is not only used for design, but it can also be used to verify the design by making a testbench.

Verification is a crucial step in any project as it tells if the block works as intended or not.

While Verilog is mainly used in design, system Verilog is used in verification more as it has more verification features than Verilog

Basic Syntax & Structure

Basic Syntax & Structure

Lexical Conventions

Verilog syntax defines the lexical conventions and structural rules for writing hardware description code. Understanding these fundamental elements--from operators and number formats to identifiers and keywords--is essential for writing correct, readable RTL designs that synthesize properly.

There is multiple of rules to write a correct code:

- Each line should end with a semi-colon “;”.
- Verilog is case sensitive language.
- To write a single line comments we start the line with “//”.
- To write multiple line comments we start with “/*” and end with “*/”.



```
1 // single line comment
2
3 /*
4 multi
5 lines
6 comment
7 */
8
9 not a comment
```

Verilog - test.v

Basic Syntax & Structure

Identifiers

Identifiers are names of variables so that they can be referenced later on. They are made up of alphanumeric characters **[a-z][A-Z][0-9]**, underscores **_** or dollar sign **\$** and are case sensitive. They cannot start with a digit or a dollar sign.

There is multiple of rules to write a correct name:

- Must start with a letter **(a-z, A-Z)** or underscore **(_)**
- Can contain letters, digits, underscores, and dollar signs **(\$)**
- Cannot start with a digit or dollar sign
- Case-sensitive: signal and Signal are different
- Cannot be a reserved keyword

Verilog - Identifiers.v

```
1 //valid names
2
3 integer address; // normal name
4 integer rst_n; // can have underscore
5 integer data1; // can have a number
6 integer _signal; //can start with an underscore
7
8 //invalid names
9
10 integer 12number; // start with a number
11 integer $dollar; // can have dollar sign
12 integer data-1; // have invalid character
```

Basic Syntax & Structure

Numbers

Numbers are the way we can express the value of the signals.

We have three types of numbers:

- Decimal
- Hexa
- Binary

Decimal is the default value type.

Each number can have a defined size with a default value of **32 bits**.

```
verilog-tut - 02_numbers_sized_unsized.v

1 // the format
2 [size]'[type] [Value];
3
4 //sized numbers
5
6 3'b010;          //3-bit binary number
7 7'd10           //7-bit decimal number
8 4'hA            //4-bit hexa number
9 32'hABCD_1234    //32-bit hexa number "can use underscore to make the number readable"
10
11 //unsized numbers
12
13 10              //32-bit decimal number "default"
14 'hAB           //32-bit hexa number
15
```

Basic Syntax & Structure

Numbers

- Numbers can be negative by adding a negative sign before the width of the signal.
- Negative numbers must have a size and can't be unsigned



```
1 //Negative numbers
2
3 -4'd10 //negative 10
4
5 4'd-10 //Invalid
```

Basic Syntax & Structure

Verilog Module and ports

- A Module is a basic building design block in Verilog, and it can be an element that implements necessary functionality. It can also be a collection of lower-level design blocks.
- A module should be enclosed within module and endmodule keywords. Name of the module should be given right after the module keyword, and an optional list of ports may be declared as well.
- We can't define a module inside another module.



```
1 // Basic module syntax with port list
2 module <name> ([port_list]);    //port list is optional
3     //module body
4 endmodule
5
6 // Basic module syntax without port list
7 module <name>;
8     //module body
9 endmodule
```

Basic Syntax & Structure

Verilog Module and ports

- Ports are a set of signals that act as inputs and outputs to a particular module and are the primary way of communicating with it.
- Type of ports are:
 - **Input port:** The design module can only receive values from outside using its input ports.
 - **Output port:** The design module can only send values to the outside using its output ports.
 - **Inout port:** The design module can either send or receive values using its inout ports **“bidirectional”**.

```
Verilog - module.v

1 // Basic module syntax with port list
2 module <name> ([port_list]);    //port list is optional
3     //module body
4 endmodule
5
6 // Basic module syntax without port list
7 module <name>;
8     //module body
9 endmodule
```

Basic Syntax & Structure

Verilog Module and ports

When we work on comparison or arithmetic operations the ports has to be signed as it behave differently in comparisons and arithmetic from the unsigned ports.

To define the port size, we define the range in square brackets before the port name.

We can't have two ports of the same name.

```
Verilog - module.v

1 //example for the module
2 module alu (
3     input [3:0] a, //four bits signal -from 0 LSB to 3 MSB-
4     input [3:0] b,
5     output [3:0] c,
6     output carry_out //one bit signal
7 );
8
9 // alu implementation
10 endmodule
11
12 //port type can be defined in the module itself
13 module alu (a, b, c, carry_out);
14
15     input [3:0] a,
16     input [3:0] b,
17     output [3:0] c,
18     output carry_out
19
20 // alu implementation
21 endmodule
22
23 //for correct arithmetic operation operands have to be signed
24 module alu (
25     input signed [3:0] a,
26     input signed [3:0] b,
27     output signed [3:0] c,
28     output carry_out
29 );
30
31 // alu implementation
32 endmodule
```

Basic Syntax & Structure

Verilog Module and ports

We can call a module inside another module.

We do this when a top module pass the values to the port of another module.

There is two ways to instantiate a module:

- Connecting a port list in an ordered manner.
- Connecting a port list by name.

```
Verilog - module.v

1 <module name> <instantiation name> [port definition];
2
3 module add (
4     input signed [3:0] first,
5     input signed [3:0] second,
6     output signed [3:0] out,
7     output carry
8 );
9
10 //addition implementation
11 endmodule
12
13 module alu (
14     input signed [3:0] a,
15     input signed [3:0] b,
16     output signed [3:0] c,
17     output carry_out
18 );
19
20 // 1. Instantiation by Position (Order)
21 add add_inst1 (a, b, c, carry_out);
22 // Order of the ports IS important here.
23 // They must exactly match the order defined in the `add` module:
24 // 1st: first, 2nd: second, 3rd: out, 4th: carry.
25 // The names of the signals being passed do not have to be the same.
26
27 // 2. Instantiation by Name
28 add add_inst2 (.first(a), .second(b), .out(c), .carry(carry_out));
29 // Order of the ports is NOT important here.
30 // We explicitly state which signal connects to which port.
31 // For example, this would also be perfectly valid and work exactly the same:
32 // add add_inst2 (.out(c), .carry(carry_out), .first(a), .second(b));
33
34 // alu implementation
35 endmodule
```

Data Types

Verilog Data Types

The values that can be assigned to any Verilog signal are:

1. 0 which represent a low voltage.
2. 1 which represent a high voltage
3. X which represent an unknown state “value not defined”
4. Z which represent high impedance state “signal not connected”

X or x means that the value is simply unknown at the time and could be either 0 or 1. This is quite different from the way X is treated in Boolean logic, where it means "don't care".

As with any incomplete electric circuit, the wire that is not connected to anything will have a high-impedance at that node and is represented by Z or z.

Data Types

Verilog Data Types

There are multiple data types in Verilog; where the most popular one is “Wire”.

The wire is similar to the electrical wire that is used to connect two components on a breadboard.

It is used to carry data and connect signals in our design.

It can be a single-bit signal or multiple-bit signal.

Wire is the default data type for all signals, and we can't have two signals with the same name.

```
Verilog - data_types_wire.v
1  module mux (
2      input wire [3:0] first, // 4-bits wire input
3      input [3:0] second,    // 4-bits wire input "by default"
4      output wire [3:0] out,  // 4-bits wire output
5  );
6
7  wire [1:0] signal; // 2-bits wire internal signal
8
9  //addition implementation
10 endmodule
```

Data Types

Verilog Data Types

There are other data types like:

Integer

An integer is a general purpose variable of 32-bits wide that can be used for other purposes while modeling hardware and stores integer values.

Time

A time variable is unsigned, 64-bits wide and can be used to store simulation time quantities for debugging purposes. A realtime variable simply stores time as a floating point quantity.

real

A real variable can store floating point values and can be assigned the same way as integer and reg.

Data Types

Verilog Data Types

```
Verilog - data_types_other_types.v

1  module testbench;
2      integer    int_a;           // Integer variable (32-bit signed)
3      real       real_b;         // Real variable (64-bit floating point)
4      time       time_c;         // Time variable (64-bit unsigned)
5
6      initial begin
7          int_a = 32'hcafe1234;    // Assign hexadecimal value to integer
8          real_b = 0.1234567;      // Assign floating point value to real
9
10         #20;                    // Advance simulation time by 20 units
11         time_c = $time;          // Capture current simulation time ($time returns time type)
12
13         // Print all variables using $display with appropriate format specifiers
14         $display ("int_a  = 0x%0h", int_a); // %0h = hexadecimal format
15         $display ("real_b = %0.5f", real_b); // %0.5f = 5 decimal places
16         $display ("time_c = %0t", time_c);  // %0t = time format
17     end
18 endmodule
```

```
Verilog - transcript.log

1  # int_a      = 0xcafe1234
2  # real_b     = 0.12346
3  # time_c     = 20
4  # End time: 23:14:27 on Jun 28,2026, Elapsed time: 0:00:09
```

Data Types

Verilog scalar and vector

A vector is any signal that has multiple bits.

Any bit in a vectored variable can be individually selected and assigned; This is called as a bit-select. If the bit-select is out of bounds or the bit-select is x or z, then the value returned will be x.

```
Verilog - data_types_part_select.v

1 module data_types_part_select;
2     wire [7:0] signal; //8-bits signal
3
4     assign signal[0] = 0; //assign LSB
5     assign signal[1] = 1; //assign bit number 1
6     assign signal[7:2] = 6'b111000; //assign bits from 2 to 7
7
8     initial begin
9         $monitor ("signal = %0b",signal); //display the signal
10    end
11
12 endmodule
```

```
Verilog - transcript.log

1 # Loading work.data_types_part_select(fast)
2 # signal = 11100010
3 # End time: 23:36:54 on Jun 28,2026, Elapsed time: 0:00:05
```

Verilog Operators

Verilog operators enable mathematical computations, logical comparisons, and bit manipulations essential for describing digital circuit behavior.

Every Verilog operator produces a result with a specific bit-length, which depends on the operator type, not just the operands.

Category	Examples	Bit Length
Bitwise	$\sim A$, $+A$, $-A$ $A \& B$, $A B$, $A \sim^{\wedge} B$, $A \wedge^{\sim} B$	$L(A)$ $\text{MAX } (L(A), L(B))$
Logical	$!A$, $A \&\&B$, $A \parallel B$	1 bit
Reduction	$\&A$, $\sim \&A$, $ A$, $\sim A$, $\wedge^{\sim} A$, $\sim^{\wedge} A$	1 bit
Relational	$A == B$, $A != B$, $A > B$, $A < B$ $A >= B$, $A <= B$ $A === B$, $A! == B$	1 bit
Arithmetic	$A + B$, $A - B$, $A * B$, A / B $A \% B$	$\text{MAX } (L(A), L(B))$
Shift	$A << B$, $A >> B$	$L(A)$
Concatenate	$\{A, \dots, B\}$	$L(A) + \dots + L(B)$
Replication	$\{B\{A\}\}$	$B * L(A)$
Condition	$A ? B : C$	$\text{MAX } (L(B), L(C))$

Verilog Operators

Example

```
verilog - Arrays_and_Memories.v

module operators_example;

    reg [3:0] A = 4'b1010;    // A = 10
    reg [3:0] B = 4'b0110;    // B = 6

    initial begin

        // ---- Bitwise ----
        $display("~A      = %b", ~A);        // ~A      = 0101    (5)
        $display("A & B   = %b", A & B);      // A & B   = 0010    (2)
        $display("A | B   = %b", A | B);      // A | B   = 1110    (14)
        $display("A ^ B   = %b", A ^ B);      // A ^ B   = 1100    (12)

        // ---- Logical ----
        $display("A&&B    = %b", A && B);      // A&&B    = 1
        $display("A||B    = %b", A || B);      // A||B    = 1
        $display("!A     = %b", !A);          // !A      = 0

        // ---- Reduction ----
        $display("&A      = %b", &A);          // &A      = 0    (AND of 1,0,1,0)
        $display("|A      = %b", |A);          // |A      = 1
        $display("^A      = %b", ^A);          // ^A      = 0

        // ---- Relational ----
        $display("A>B     = %b", A > B);      // A>B     = 1
        $display("A==B    = %b", A == B);     // A==B    = 0

        // ---- Arithmetic ----
        $display("A+B     = %d", A + B);      // A+B     = 16
        $display("A-B     = %d", A - B);      // A-B     = 4
        $display("A*B     = %d", A * B);      // A*B     = 60
        $display("A%B     = %d", A % B);      // A%B     = 4

        // ---- Shift ----
        $display("A<<1    = %b", A << 1);      // A<<1    = 0100    (10100 truncated to 4 bits)
        $display("A>>1    = %b", A >> 1);      // A>>1    = 0101    (5)

        // ---- Concatenation ----
        $display("{A,B}   = %b", {A, B});      // {A,B}   = 10100110    (166)

        // ---- Replication ----
        $display("{2{A}}  = %b", {2{A}});      // {2{A}}  = 10101010    (170)

        // ---- Conditional ----
        $display("sel?A:B= %d", 1'b1 ? A : B); // sel?A:B = 10 (A chosen)

    end

endmodule
```

Verilog Arrays and Memories

Verilog Arrays and Memories

Arrays

In Verilog, an array is a data structure that allows you to group multiple data elements of the same data type into a single collection. Arrays provide a convenient way to work with and manipulate multiple data values using a single name.

Arrays are allowed in Verilog for **reg**, **wire**, **integer** and **real** data types. When the elements of an array are registers, the array is called a **memory**.

Verilog also supports arrays with more than one index dimension, useful for modeling structures like register files.

Each additional pair of brackets after the variable name adds another array dimension (**as opposed to brackets before the name, which set bit-width**).

```
verilog-tut - arrays.v

1  /* Initialization */
2  reg arr1 [11:0];           // arr is a scalar reg array of depth=12, each 1-bit wide
3  wire [0:7] arr2 [3:0]      // arr is an 8-bit vector net with a depth of 4
4  reg [7:0] arr3 [0:1][0:3]; // arr is a 2D array rows=2,cols=4 each 8-bit wide
5
6  //You can initialize the elements of an array during declaration or later in the code
7  reg [3:0] regfile [0:7];   // 8 registers, each 4 bits wide
8  regfile[2] = 4'b1010;      // write register 2
9
10 arr1 = 0;                  // Illegal - All elements can't be assigned in a single go
11
12 arr2[0] = 8'ha2;           // Assign 0xa2 to index=0
13 arr3[1][2] = 8'hdd;        // Assign 0xdd to rows=1 cols=2
```

Verilog Arrays and Memories

Memories

Memories refer to digital storage elements that are used to store and retrieve data within a digital circuit. often used to implement various types of data storage, such as **RAM** (Random-Access Memory) and **ROM** (Read-Only Memory).

A memory is declared with two ranges:
one for the **width** of each word, and one for the **number of words**.

A memory cannot be initialized directly in its declaration the way a scalar can (reg [7:0] mem [0:15] = 8'h00; is not valid).



verilog - Arrays_and_Memories.v

```
reg [msb:lsb] mem_name [upper_addr:lower_addr]; //Declaring a Memory
reg [7:0] mem [0:15];    // 16 words, each 8 bits wide

/*
[7:0] → each word is 8 bits
[0:15] → there are 16 words, addressed 0 through 15
*/
```

Verilog Arrays and Memories

Memories

Accessing a Memory

mem[word][bit]

Only one word can be read or written per statement — you can't assign to a range of words the way you can with a vector's bit-range (**e.g. mem[2:5] is not valid**).

```
● ● ●   verilog - Arrays_and_Memories.v

//Accessing a Memory Word
mem[4] = 8'hAA;           // write to word 4
$display("%h", mem[4]);    // read word 4

// Accessing a Bit Inside a Word
$display("%b", mem[4][3]); // bit 3 of word 4
```

Out-of-range indexing (accessing an address outside the declared bounds) **returns x**

Verilog Arrays and Memories

Memories

Initializing a Memory

Memories can be initialized element by element, often using a loop inside an initial block.

verilog - Arrays_and_Memories.v

```
integer i;
initial begin
    for (i = 0; i < 16; i = i + 1)
        mem[i] = i;
end
```

Verilog also provides **\$readmemh** and **\$readmemb** system tasks to load memory contents directly from a file — commonly used to preload ROM or RAM models for simulation.

verilog - Arrays_and_Memories.v

```
module rom_with_file(
    input [2:0] addr,
    output [1:0] data
);

    reg [1:0] rom [0:7];

    initial
        $readmemb("truth_table.mem", rom);

    assign data = rom[addr];
endmodule
```

Verilog Parameters

Parameter

A parameter is a constant value declared inside a module, used to make code more flexible and reusable.

parameter parameter_name = value;

verilog - Arrays_and_Memories.v

```
module design_ip
    #(parameter BUS_WIDTH=32,
      parameter DATA_WIDTH=64) (

        input [BUS_WIDTH-1:0] addr,
        // Other port declarations
    );
```

verilog-tut - parameters.v

```
1 module design_ip
2     #(parameter BUS_WIDTH=32,
3       parameter DATA_WIDTH=64) (
4
5         input [BUS_WIDTH-1:0] addr,
6         // Other port declarations
7     );
8
9     parameter MSB = 7;    // MSB is a parameter with a constant value 7
10    parameter REAL = 4.5; // REAL holds a real number
11
12    parameter FIFO_DEPTH = 256,
13      MAX_WIDTH = 32; // Declares two parameters
14
15    parameter [7:0] f_const = 2'b11; // 2 bit value is converted to 8 bits; 8'b11
16 endmodule
```

Verilog Parameters

Parameter

Overriding Parameters at Instantiation

A parameter's default value can be overridden when the module is instantiated, without touching the module's internal code.

There are two ways to override a parameter:

- **Ordered** (#() before instance name)
- **Named** (.PARAM(value)), similar to named port connections

```
verilog-tut - parameters_override_instantiation.v

1 //Overriding Parameters at Instantiation
2 counter #(.WIDTH(16)) big_counter (.count(count_out));
3
4 module tb;
5
6     // Module instantiation override
7     design_ip #(.BUS_WIDTH(64), .DATA_WIDTH(128)) d0 ( [port list]);
8
9     // Use of defparam to override
10    defparam d0.FIFO_DEPTH = 128; //commonly used in testbench simulations to quickly update the design parameters without having to reinstantiate the module.
11
12 endmodule
13
```

Verilog Parameters

localparam

localparam declares a constant just like parameter, but it cannot be overridden at instantiation.

It's used for values that depend on other parameters or represent internal constants (like state names) that should never be changed from outside the module.

```
verilog-tut - parameters_localparam.v

1  module adder #(parameter WIDTH = 8, PARAM_LENGTH = 4) (
2      input wire [WIDTH-1:0] a,
3      input wire [WIDTH-1:0] b,
4      output wire [WIDTH-1:0] sum
5  );
6
7      // Assign a part-select of original parameter
8      localparam LENGTH = 2 + PARAM_LENGTH[1:0];
9
10     assign sum = a + b;
11
12     initial begin
13         $display("Length = %d", LENGTH);
14     end
15 endmodule
16
17 module tb;
18     // Statements
19
20     // Update parameter length to 6, and localparam should get 2 + 2'b10 = 4
21     adder #(.WIDTH(16), .PARAM_LENGTH(6)) add_instance2 ( ... );
22 endmodule
```

Procedural Blocks

Procedural Blocks

A procedural block is a section of code that executes **statement by statement, in order** — like a normal program. It represents a **behavior** you want to describe, not a physical wire.

"Procedural" doesn't mean "sequential hardware" . it just means the *code* executes step-by-step. Whether it becomes combinational or sequential *hardware* depends on the sensitivity list and how signals are assigned.



verilog - procedural_blocks.v

```
// Dataflow – continuous, always active
assign y = a & b;           // y updates instantly whenever a or b changes

// Procedural – executes when triggered
always @(*) begin
    y = a & b;               // same logic, but written procedurally
end
```

Procedural Blocks

Initial Block

- The initial block executes exactly once, starting at simulation time 0. It does not repeat and does not respond to signal changes. Setting up initial conditions and stimulus in testbenches (clock generation, reset pulses, applying input vectors).
- **Not synthesizable** — no real hardware equivalent, since actual circuits don't have a "run once at time 0" behavior. Used only in simulation/testbench code, never in synthesizable RTL.
- An initial block is started at the beginning of a simulation at time 0 unit. This block will be executed only once during the entire simulation. Execution of an initial block finishes once all the statements within the block are executed.

```
verilog - procedural_blocks.v

initial begin
    clk = 0;           // initialize clock
    reset = 1;         // assert reset at time 0
    #10 reset = 0;      // deassert reset after 10 time units - Can contain delays (#) to schedule events over simulation time.
    #5 data_in = 8'hFF; // apply test input at time 15
end

initial begin
    clk = 0;           // block A: starts at time 0
end

initial begin
    $display("Simulation started"); // block B: also starts at time 0
end
```

Procedural Blocks

Always Block

- The always block executes repeatedly — every time a signal in its sensitivity list changes. Unlike initial, it never stops running for the rest of simulation.
- An always block can be used to realize combinational or sequential elements. A sequential element like flip flop becomes active when it is provided with a clock and reset. Similarly, a combinational block becomes active when one of its input values change.
- These hardware blocks are all working concurrently independent of each other. The connection between each is what determines the flow of data. To model this behavior, an always block is made as a continuous process that gets triggered and performs some action when a signal within the sensitivity list becomes active

```
verilog - procedural_blocks.v

//Syntax:
always @(sensitivity_list) begin
    // statements execute whenever a listed signal changes
end

//Two common forms:
always @(posedge clk) // triggers only on rising edge of clk -> sequential logic
always @(*)           // triggers whenever ANY input inside changes -> combinational logic
```

Procedural Blocks

Always Block

- A sensitivity list is the expression that defines when the always block should be executed and is specified after the @ operator within parentheses (). This list may contain either one or a group of signals whose value change will execute the always block.
- Complete sensitivity lists are **critical** - missing inputs cause simulation-synthesis mismatches; Best practice use @(*) for automatic inference in combinational blocks
- The always block repeats continuously throughout the duration of a simulation. The sensitivity list brings along a certain sense of timing i.e. whenever any signal in the sensitivity list changes, the always block is triggered. If there are no timing control statements within an always block, the simulation will hang because of a zero-delay infinite loop .



verilog - procedural_blocks.v

```
// No sensitivity list and no delay - simulation hangs
// always block is started at time 0 units But when is it supposed to be repeated?
// There is no time control, and hence it will stay and be repeated at 0 time units only. This continues
// in a loop and simulation will hang
always clk = ~clk;

// CORRECT for testbenches: Explicit delay creates clock with 20ns period
// Toggles every 10ns: high for 10ns, low for 10ns
always #10 clk = ~clk;
```

Procedural Blocks

Always Block

- Any signal assigned inside a procedural block must be **declared as reg**, not wire.
- A procedural block doesn't "**wire**" a value, it assigns it as an action, at a specific point in the code's execution. Verilog needs a data type that can hold that assigned value until the next assignment — that's reg.
- **reg** does not always mean a real hardware register/flip-flop. It's just a storage variable required by procedural assignment. Whether it becomes a flip-flop or plain combinational logic depends on how it's used
- **Best Practice:** Use separate always blocks for combinational and sequential logic. Never mix edge and level sensitivity in one block; synthesis tools reject that.

```
verilog - procedural_blocks.v

//Combinational Element Design Example
module combo ( input  a,
               input  b,
               input  c,
               input  d,
               output reg o); // Output must be 'reg' type for procedural assignment

// Level-sensitive always block: Triggers when ANY input changes
// This infers combinational logic (no clock, no memory elements)
always @ (a or b or c or d) begin
    // Combinational expression: o = NOT((a AND b) OR (c XOR d))
    o = ~((a & b) | (c^d));
end

endmodule
```

```
verilog - procedural_blocks.v

// Sequential Element Design Example
module tff (input  d,      // Data input: toggles output when high
           clk,          // Clock signal: triggers on rising edge
           rstn,         // Active-low asynchronous reset
           output reg q); // Output register (must be reg in always block)

// Edge-sensitive always block: Triggered on posedge clk OR negedge rstn
// This implements asynchronous reset behavior
always @ (posedge clk or negedge rstn) begin
    if (!rstn)
        q <= 0; // Asynchronous reset: Immediately set q to 0
    else
        // Synchronous behavior: Only evaluated at clock edges
        if (d)
            q <= ~q; // Toggle output when d=1
        else
            q <= q; // Hold output when d=0
    end
endmodule
```

Procedural Blocks

Block Statements

- There are ways to group a set of statements together so they act as a single statement — needed because constructs like **if** or **always** only take one statement as their body. These groups are called block statements.
- There are two kinds of block statements: sequential (**begin – end**) and parallel (**fork – join**). Block statements enable structured control flow, scoped variable declarations, and concurrent execution modeling.
- **begin – end**: statements execute in the order they're written; each one finishes before the next starts — just like a normal sequential program.
- **fork – join**: statements all start together and run concurrently; the block finishes only once the **last** one completes. Used almost exclusively in testbenches — not synthesizable.

```
verilog - block_statements.v

module block_demo;
    reg [3:0] a, b;

    // Sequential block: order matters
    initial begin
        a = 4'd0;
        a = a + 1;    // runs after the line above
        a = a + 1;    // a is now 2
    end

    // Parallel block: all statements start together
    initial fork
        #5 b = 4'd1;    // both scheduled at time 0,
        #2 b = 4'd2;    // finishes when the last (#5) is done
    join
endmodule
```

Verilog Scheduling Semantics

Verilog Scheduling Semantics

Verilog simulation doesn't execute statements top-to-bottom like a normal program. Instead, the simulator maintains an **event queue** organized into time steps, and within each time step, into ordered **regions**. Understanding this ordering is essential for knowing why blocking (**=**) and non-blocking (**<=**) assignments behave differently, and why race conditions happen.

The Four Main Regions (per simulation time step)

For each simulation time, events are processed in this order:

Active region

Blocking assignments (**=**) execute here, immediately, in the order they're encountered.

Continuous assignments (**assign**) are evaluated here.

`$display` statements execute here.

Right-hand sides (**RHS**) of non-blocking assignments (**<=**) are evaluated here — but the actual update to the left-hand side (**LHS**) is deferred.

Events in this region can be processed in any order relative to each other (if multiple things are ready at the same time), which is a common source of race conditions between blocks.

Inactive region

Statements with an explicit **#0** delay are scheduled here — executed after all active-region events for the current time step, but still at the same simulation time.

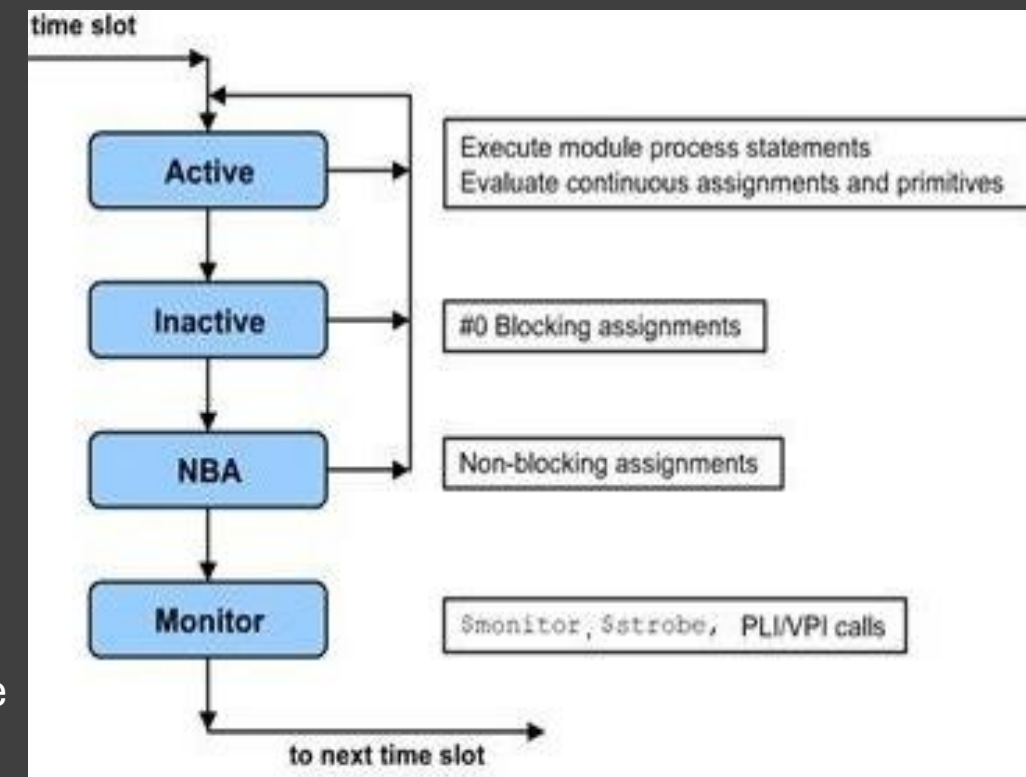
NBA (Non-Blocking Assignment) region

This is where the LHS updates for all non-blocking assignments (**<=**) actually happen, using the RHS values that were computed back in the Active region.

This delay — evaluate now, update later — is exactly why **<=** is used for sequential logic: it lets all flip-flops in a design sample their inputs "**simultaneously**," using old values, before any of them update.

Postponed region

\$strobe and **\$monitor** execute here, after everything else for the current time step has settled — so they always print the final, settled values of signals, never an intermediate race-prone value.

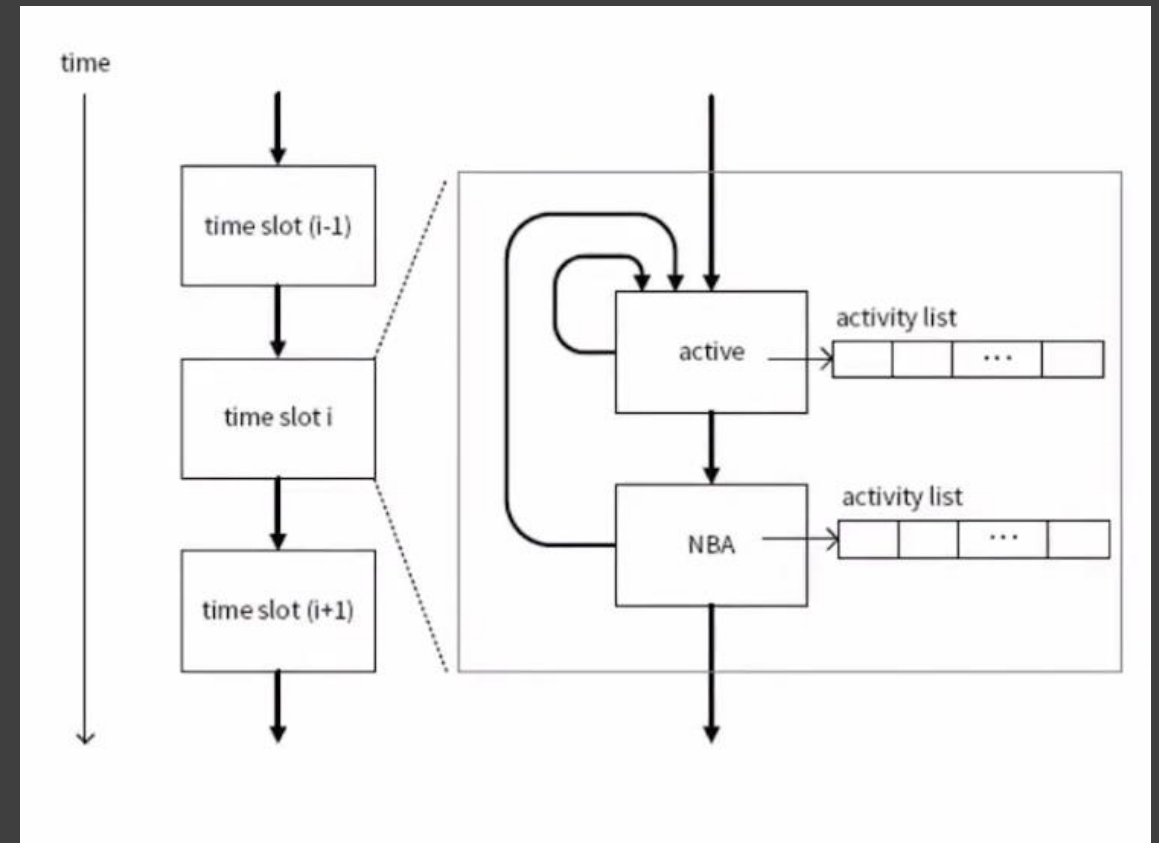


Blocking & Non-Blocking Assignments

Blocking & Non-Blocking Assignments

Why Scheduling Order Matters

- Simulators split each time step into regions. Blocking assignments execute and write in the **active** region, right away, in the order they're written.
- Non-blocking assignments evaluate their right-hand side in the active region too, but defer the actual write to the **NBA update** region — after all active-region events finish.
- That's why swapping two registers works with `<=` but not with `=`: the blocking version overwrites **a** before **b** gets to read the old value.



verilog - swap_blocking.v

```
// Blocking - BROKEN swap
always @(posedge clk) begin
    a = b;
    b = a;    // reads the NEW a,
end          // not the original value

// Result: a == b == old_b
// The original value of a is lost.
```



verilog - swap_nonblocking.v

```
// Non-blocking - CORRECT swap
always @(posedge clk) begin
    a <= b;
    b <= a;    // reads the OLD a,
end           // scheduled, not written yet

// Result: a and b are swapped,
// exactly like two real flip-flops.
```

Blocking & Non-Blocking Assignments

Blocking Assignment (=)

- A blocking assignment uses **=**. It executes and updates its target **immediately**, in program order, before the next statement in the block runs.
- Behaves like an assignment in a normal programming language — statement N+1 always sees the new value written by statement N.
- Mainly used to model **combinational logic** inside procedural blocks, where order of evaluation should mirror the order of the equations.



verilog - blocking_assign.v

```
module comb_logic (  
    input wire a, b, c,  
    output reg y  
);  
    reg tmp;  
  
    always @(*) begin  
        tmp = a & b;    // evaluated first  
        y    = tmp | c; // sees the NEW value of tmp  
    end  
endmodule
```

Blocking & Non-Blocking Assignments

Non-Blocking Assignment (<=)

- A non-blocking assignment uses `<=`. The right-hand side is evaluated **now**, but the target isn't updated until every non-blocking assignment in the current time step has been evaluated.
- All non-blocking updates in a time step appear to happen **simultaneously** — this mirrors how real flip-flops all latch their input at the same clock edge.
- Used to model **sequential (clocked) logic**.



verilog - nonblocking_assign.v

```
module shift_reg (  
    input wire clk,  
    input wire d,  
    output reg q0, q1, q2  
);  
    always @(posedge clk) begin  
        q0 <= d;    // all three right-hand sides  
        q1 <= q0;    // are read using OLD values,  
        q2 <= q1;    // then updated together  
    end  
endmodule
```

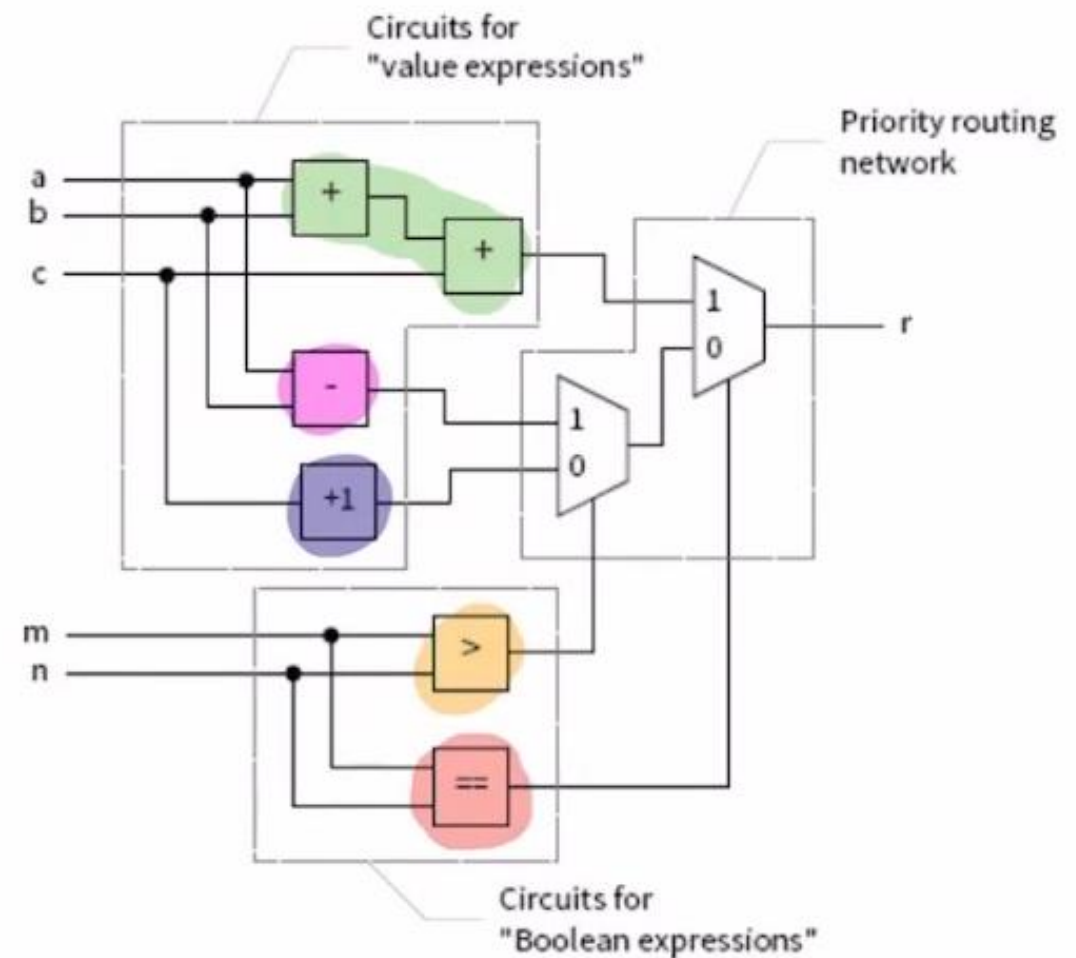
Verilog Conditional Statements

Verilog Conditional Statements

if / else if / else

- Standard conditional branching inside a procedural block (**initial/always**). A condition is evaluated as a boolean — any nonzero value counts as true.
- Chained **else if** checks multiple conditions **in priority order**
- In combinational logic, every branch must assign every output — otherwise a latch gets inferred (covered a few slides ahead).

```
if (m == n)
    r = a + b + c;
else if (m > n)
    r = a - b;
else
    r = c + 1
```



Verilog Conditional Statements

case Statement

- **case** compares one expression against a list of possible values — cleaner than a long **if / else if** chain for multi-way branching.
- **default** catches any value not explicitly listed. **Always include it** in combinational logic to avoid an inferred latch.
- Unlike independent **if** conditions, every **case** branch is checked against the exact same expression.



verilog - case_stmt.v

```
always @(*) begin
  case (sel)
    2'b00: y = a;
    2'b01: y = b;
    2'b10: y = c;
    default: y = d;
  endcase
end
```

```
logic [1:0] sel;
```

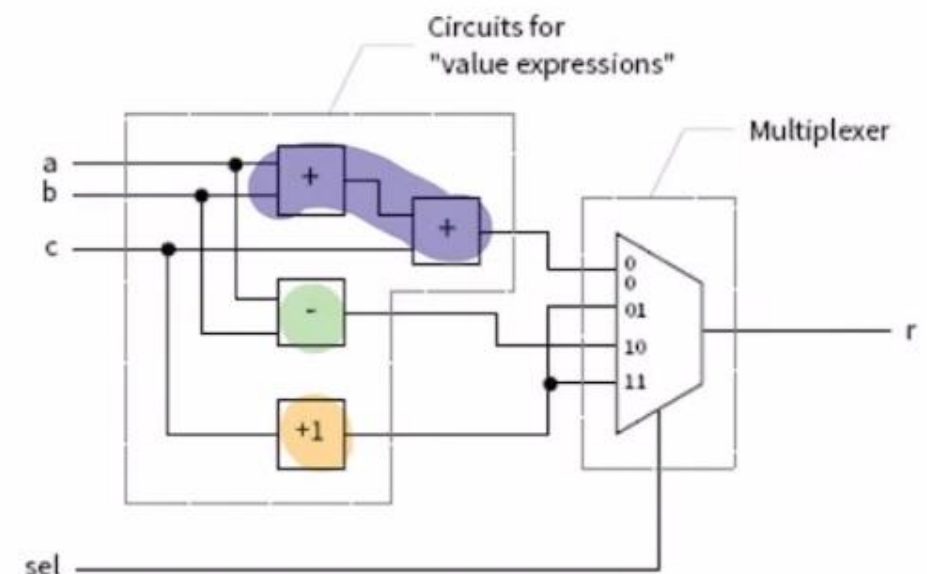
```
case (sel)
```

```
  2'b00: r = a + b + c;
```

```
  2'b10: r = a - b;
```

```
  default: r = c + 1; //all other cases
```

```
endcase
```



Verilog Conditional Statements

casex / casez

- **casez** treats bits that are **z** in the case expression or a case item as **don't-cares**.
- **casex** treats both **x** and **z** as don't-cares — more permissive, and more error-prone in simulation.
- Both are commonly used for priority encoders and instruction decoders, where some bits don't matter for a given match.
- **casex** can silently match values that are actually unknown (**x**) during simulation — prefer **casez** with explicit **?** don't-cares.



verilog - casez_priority.v

```
always @(*) begin
  casez (req)
    4'b1???: grant = 4'b1000; // highest priority
    4'b01??: grant = 4'b0100;
    4'b001?: grant = 4'b0010;
    4'b0001: grant = 4'b0001;
    default: grant = 4'b0000;
  endcase
end
```

Verilog Conditional Statements

Priority Logic & Full/Parallel Case

- If a **case/if-else** chain doesn't cover every possible input combination, the synthesizer infers a **latch** to hold the output's previous value — almost always unintended.
- Always add a **default** branch (or a final **else**) in combinational logic, even if it just reasserts a safe value.
- **if-else** chains synthesize as **priority logic** — the first match wins, so deeper conditions cost more delay. A **case** with non-overlapping values can synthesize flatter, faster logic.

verilog - latch_vs_fixed.v

```
// BAD: missing default -> inferred latch
```

```
always @(*) begin
```

```
    case (sel)
```

```
        2'b00: y = a;
```

```
        2'b01: y = b;
```

```
    endcase
```

```
end
```

```
// GOOD: every input value is covered
```

```
always @(*) begin
```

```
    case (sel)
```

```
        2'b00: y = a;
```

```
        2'b01: y = b;
```

```
        default: y = 1'b0;
```

```
    endcase
```

```
end
```

Loops & Generate

Loops & Generate

for Loop

- Repeats a statement or block a fixed number of times — behaves like a **for** loop in C once it's inside a procedural block.
- Inside procedural code the loop is "**unrolled in time**" during simulation; on its own it doesn't create parallel hardware.
- Common uses: initializing memories, packing/unpacking vectors, and writing repetitive testbench stimulus.



verilog - for_loop.v

```
integer i;  
reg [7:0] mem [0:15];  
  
initial begin  
    for (i = 0; i < 16; i = i + 1)  
        mem[i] = i;    // fills mem[0]..mem[15]  
end
```

Loops & Generate

while & repeat Loops

- **while (condition)** repeats for as long as the condition holds — re-checked before every iteration.
- **repeat (n)** executes a fixed number of times, with **n** evaluated once at the start — handy for delays and stimulus in testbenches.
- Both are far more common in **testbenches** than in synthesizable RTL.



verilog - while_repeat.v

```
// while: find the first set bit
i = 0;
while (i < 8 && !data[i])
    i = i + 1;

// repeat: generate 10 clock pulses
repeat (10) begin
    clk = ~clk;
    #5;
end
```

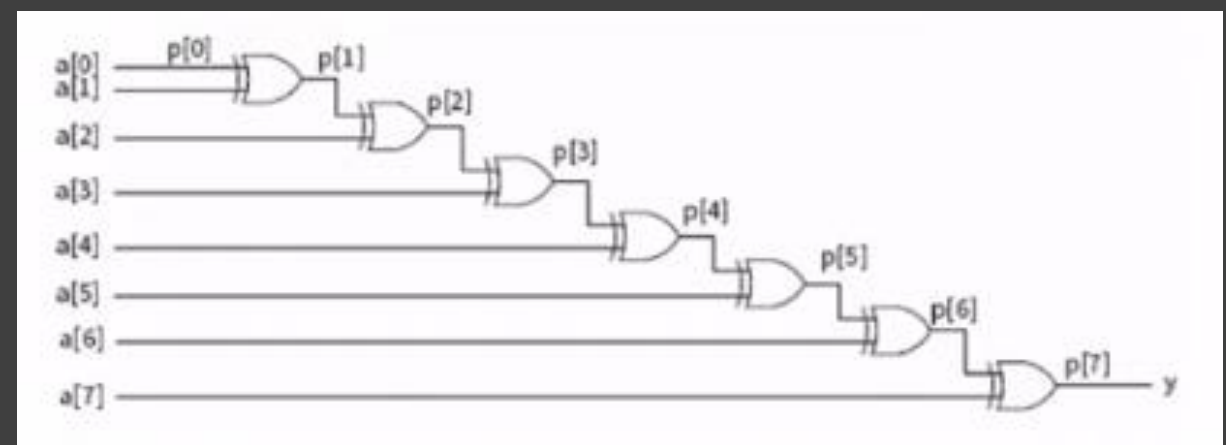
Loops & Generate

generate & genvar

- A **generate** block creates hardware structures at **elaboration time**, not simulation time — it decides *what hardware exists*, not what it does.
- **genvar** declares a loop variable used only inside **generate for** — it doesn't exist in the final hardware, only during elaboration.
- Used to replicate structural hardware, such as building an N-bit adder from N single-bit full-adder instances.

```
verilog-tut - loops_generate_genvar.v

1  module reduced_xor_gen (
2      input logic [7:0] a,
3      output logic y
4  );
5
6  logic [7:0] p;
7
8  assign p[0] = a[0];
9
10 // replicated cascading-chain structure
11 generate
12     genvar i;
13     for (i = 1; i < 8; i = i + 1)
14         assign p[i] = a[i] ^ p[i - 1];
15 endgenerate
16
17 assign y = p[7];
18
19 endmodule
```



Combinational Circuits

Combinational Circuits

Modeling Combinational Logic

- Two main styles: continuous assignment **assign** (**dataflow**) for simple expressions, and **always @(*)** for more complex conditional logic.
- Output depends only on the **current** inputs — no memory, no clock.
- **always @(*)** (or **always @***) auto-infers the complete sensitivity list from every signal read inside the block — always prefer it over listing signals by hand.

```
verilog - assign_mux.v

// Dataflow style
module mux2 (
    input wire a, b, sel,
    output wire y
);
    assign y = sel ? b : a;
endmodule
```

```
verilog - always_mux.v

// always @(*) style
module mux2 (
    input wire a, b, sel,
    output reg y
);
    always @(*) begin
        case (sel)
            1'b0: y = a;
            1'b1: y = b;
        endcase
    end
endmodule
```

Combinational Circuits

Common Building Blocks

- Multiplexer, decoder, encoder, adder/subtractor, comparator — the standard combinational "building bricks" reused across almost every design.
- These map directly onto the **if** / **case** / **casez** constructs from the previous section — a mux is a case statement, a priority encoder is a casez.



verilog - full_adder.v

```
module full_adder (  
    input wire a, b, cin,  
    output wire sum, cout  
);  
    assign sum = a ^ b ^ cin;  
    assign cout = (a & b) | (cin & (a ^ b));  
endmodule
```

Combinational Circuits

Incomplete Sensitivity & Inferred Latches

- Leaving a signal out of an explicit sensitivity list `@(a, b)` (instead of `@(*)`) means simulation won't react to a change in the missed input — a classic **simulation/synthesis mismatch**.
- Not assigning an output on every possible path (missing **else/default**) makes the synthesizer infer a **latch** to remember the old value — almost always a bug in combinational logic.
- Fix: always use `@(*)`, and assign every output on every branch (or give it a default value at the top of the block).

```
verilog - latch_bug.v

// BAD: latch inferred on y
always @(a, sel) begin
    if (sel)
        y = a;
    // no else -> y holds old value
    // 'b' missing from sensitivity list
end
```

```
verilog - latch_fixed.v

// GOOD: complete & synthesizable
always @(*) begin
    y = 1'b0;        // safe default
    if (sel)
        y = a;
    else
        y = b;
end
```

Sequential Circuits

Sequential Circuits

Modeling Sequential Logic

- Sequential logic has **memory** — its output depends on the current inputs *and* past state, stored in flip-flops/registers.
- Modeled with **always @(posedge clk)** (or **negedge clk**) — the block only executes on the active clock edge.
- Always use **non-blocking** assignments **<=** here, so all registers appear to update simultaneously, matching real flip-flop behavior.



verilog - d_flipflop.v

```
module dff (  
    input wire clk,  
    input wire d,  
    output reg q  
);  
    always @(posedge clk) begin  
        q <= d;  
    end  
endmodule
```

Sequential Circuits

Registers & Shift Registers

- A register is a flip-flop (or a bank of them) that holds a value between clock edges — declared as **reg** and written inside a clocked always block.
- A **shift register** chains registers together; each clock edge moves data one stage down the line — a natural fit for non-blocking assignment's "simultaneous update" behavior.



verilog - shift_register.v

```
module shift4 (  
    input wire clk, sin,  
    output wire sout  
);  
    reg [3:0] sr;  
  
    always @(posedge clk)  
        sr <= {sr[2:0], sin};  
  
    assign sout = sr[3];  
endmodule
```

Sequential Circuits

Sync vs Async

- **Synchronous reset:** checked only on the clock edge, alongside other logic — **always @(posedge clk)**. Cleaner timing, but needs the clock running to actually reset.
- **Asynchronous reset:** reset sits in the sensitivity list itself — **always @(posedge clk or posedge rst)** — takes effect immediately, independent of the clock. Common for power-on reset.
- Pick one style and use it **consistently** across the design — mixing styles complicates static timing analysis.



verilog - sync_reset.v

```
// Synchronous reset
always @(posedge clk) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end
```



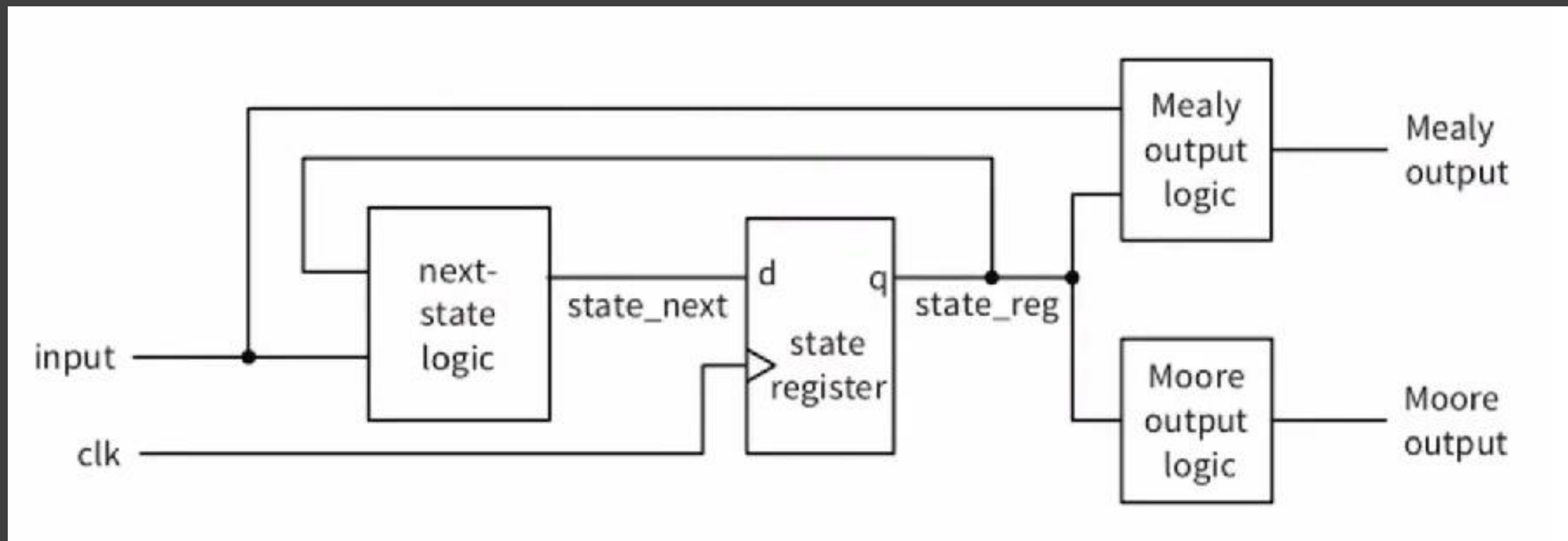
verilog - async_reset.v

```
// Asynchronous reset
always @(posedge clk or
        posedge rst) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end
```

Sequential Circuits

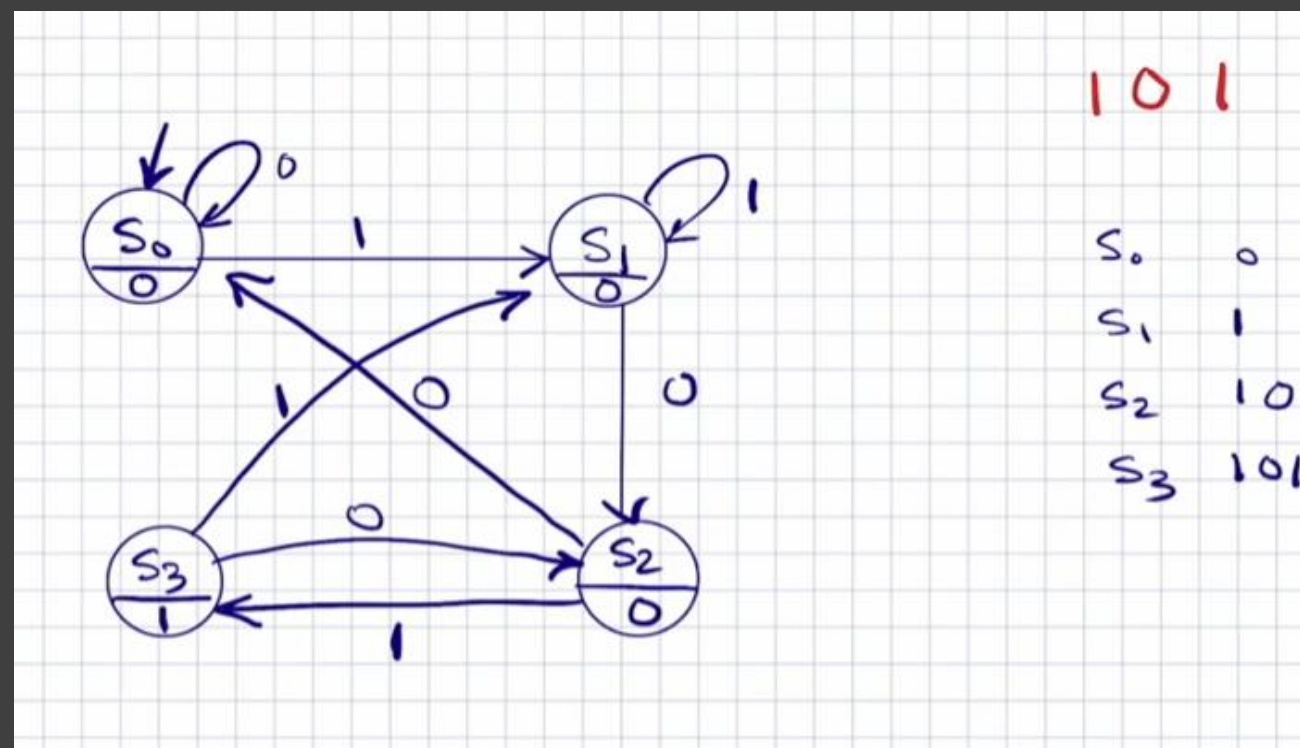
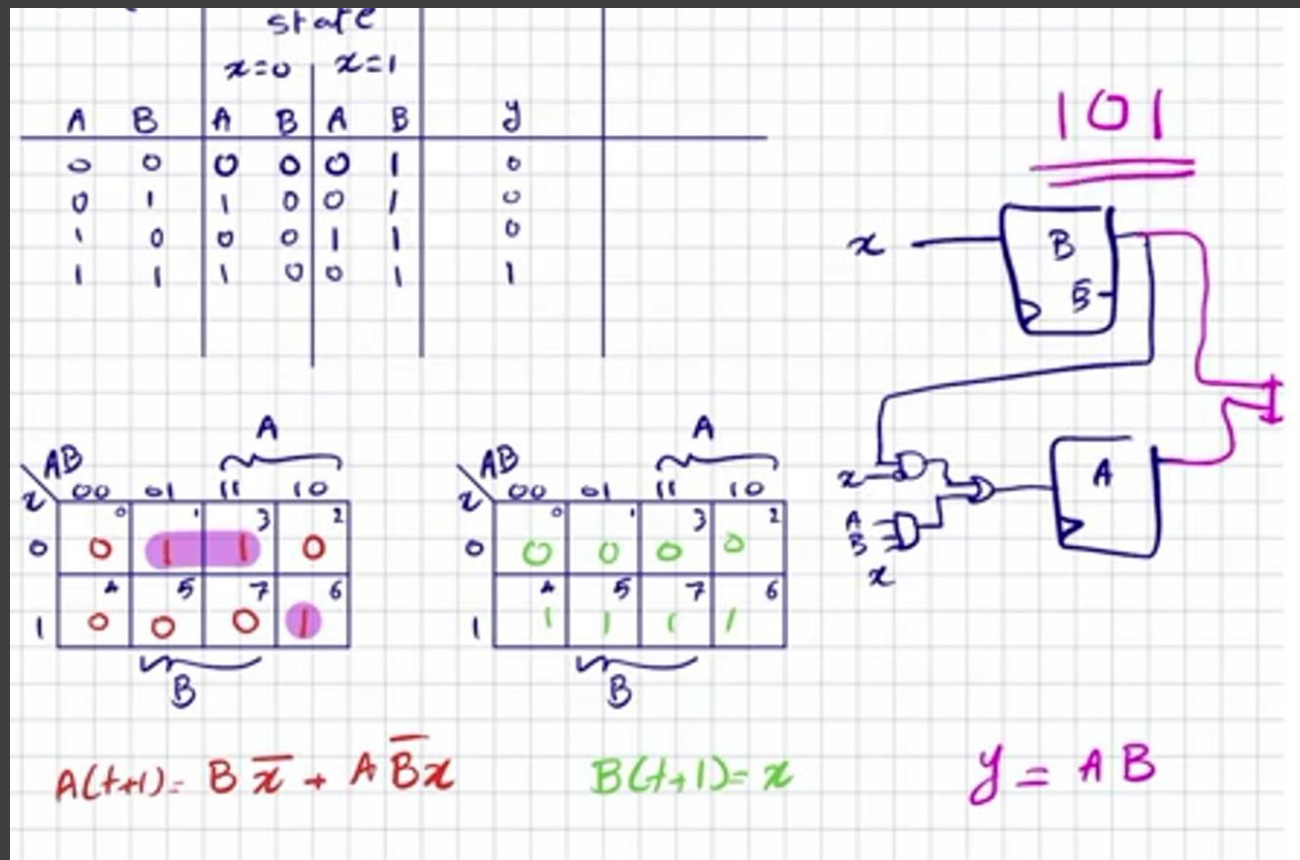
Finite State Machines (FSM)

- An FSM moves between a fixed set of named states based on inputs — usually modeled with **localparam** state encodings plus a state register.
- Common style: **two always blocks** — one sequential block updates the state register on the clock edge; one combinational block (a **case** on the current state) computes the next state and outputs.
- Splitting state-register logic from next-state logic keeps sequential and combinational concerns separated — matching the best-practice guidance from earlier.



Sequential Circuits

Finite State Machines (FSM) Examples



```

verilog - procedural_blocks.v

module moore_101_detector(
    input clk,
    input reset_n,
    input x,
    output y
);

    reg [1:0] state_reg, state_next;
    localparam s0 = 0;
    localparam s1 = 1;
    localparam s2 = 2;
    localparam s3 = 3;

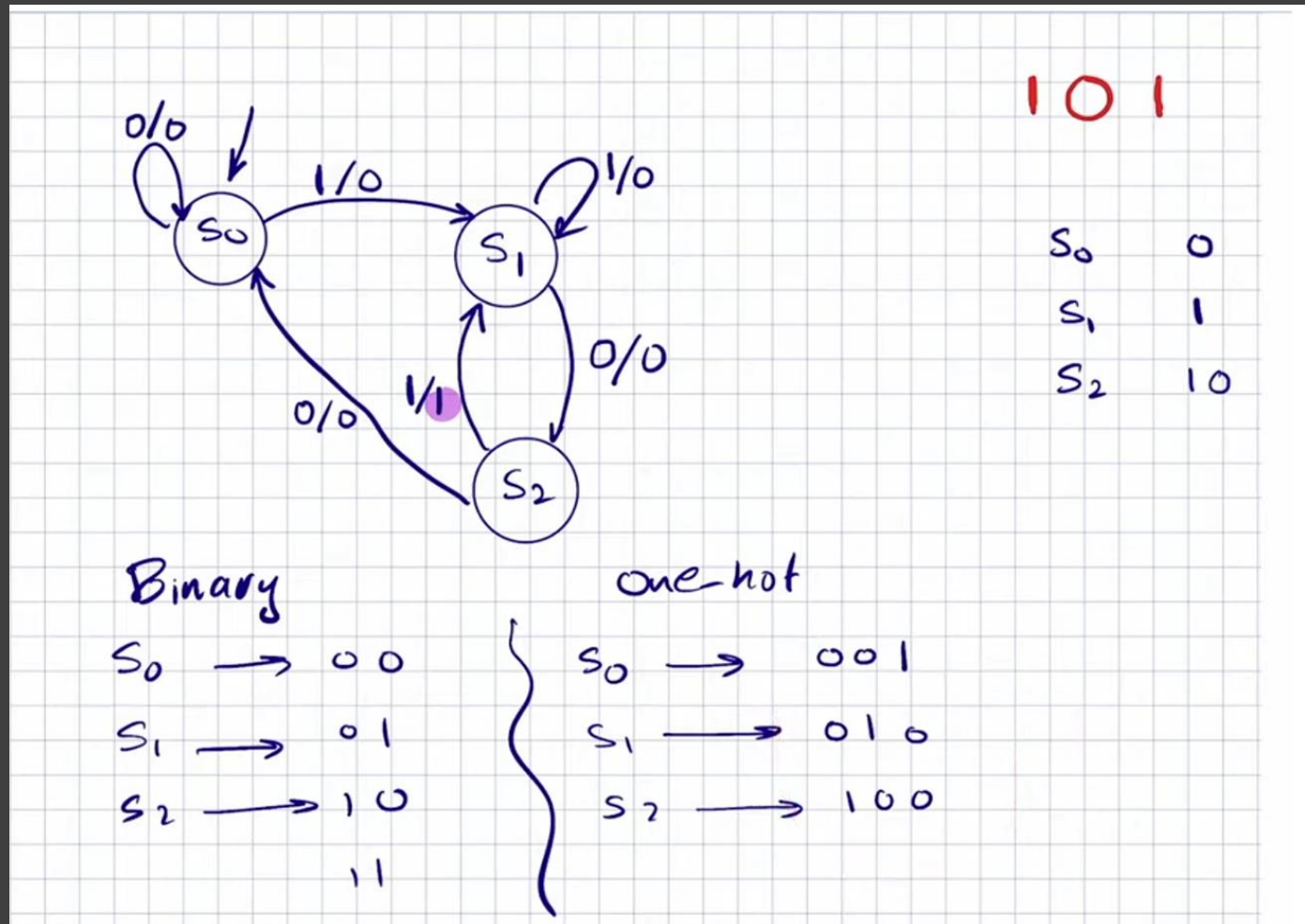
    // State register
    always @(posedge clk, negedge reset_n)
    begin
        if (~reset_n)
            state_reg <= s0;
        else
            state_reg <= state_next;
    end

    // Next state logic
    always @(*)
    begin
        case(state_reg)
            s0: if(x)
                    state_next = s1;
                else
                    state_next = s0;
            s1: if(x)
                    state_next = s1;
                else
                    state_next = s2;
            s2: if(x)
                    state_next = s3;
                else
                    state_next = s0;
            s3: if(x)
                    state_next = s1;
                else
                    state_next = s2;
            default: state_next = state_reg;
        endcase
    end

    // Output logic
    assign y = (state_reg == s3);
endmodule
  
```

Sequential Circuits

Finite State Machines (FSM) Examples



```
verilog - procedural_blocks.v

module mealy_101_detector(
    input clk,
    input reset_n,
    input x,
    output y
);

    reg [1:0] state_reg, state_next;
    localparam s0 = 0;
    localparam s1 = 1;
    localparam s2 = 2;
    localparam s3 = 3;

    // State register
    always @(posedge clk, negedge reset_n)
    begin
        if (~reset_n)
            state_reg <= s0;
        else
            state_reg <= state_next;
    end

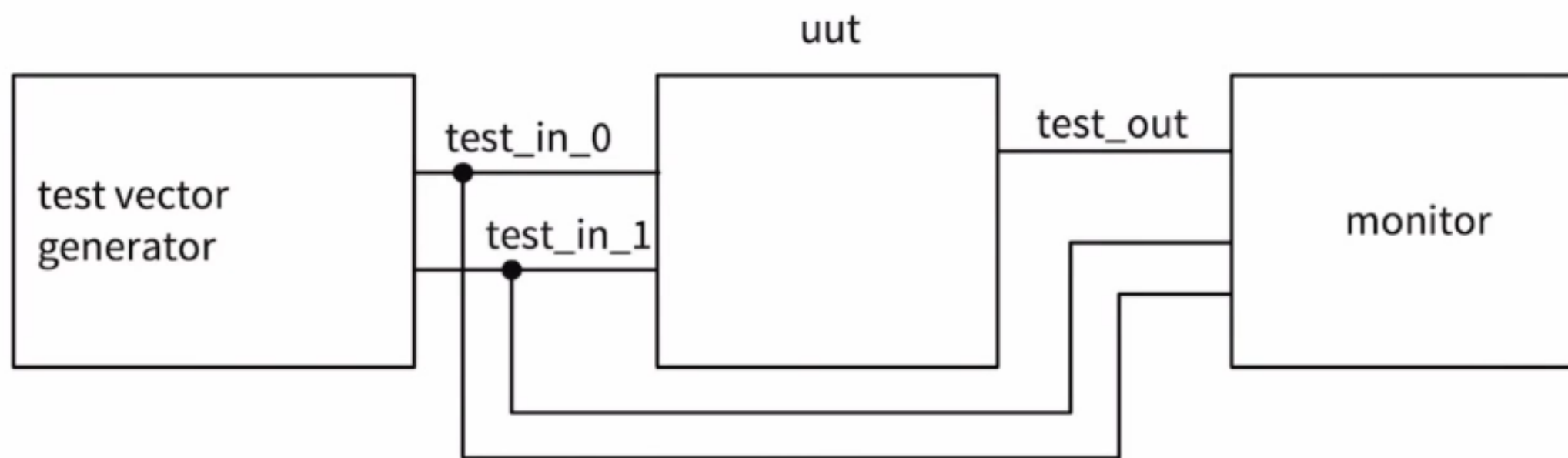
    // Next state logic
    always @(*)
    begin
        case(state_reg)
            s0: if(x)
                    state_next = s1;
                else
                    state_next = s0;
            s1: if(x)
                    state_next = s1;
                else
                    state_next = s2;
            s2: if(x)
                    state_next = s1;
                else
                    state_next = s0;
            default: state_next = state_reg;
        endcase
    end

    // Output logic
    assign y = (state_reg == s2) & x;
endmodule
```

Testbenches

Testbenches

- A Verilog testbench is a simulation environment used to verify the functionality and correctness of a digital design described in the Verilog hardware description language (HDL).
- The purpose of a testbench is to provide a way to simulate the behavior of the design under various conditions, inputs, and scenarios before actually fabricating the physical hardware. It allows designers to catch bugs, validate functionality, and optimize designs without the cost and time associated with physical prototyping.
- Testbenches are used only for simulation purposes and not for synthesis. Hence the full range of Verilog constructs like initial and system tasks like **\$display** can be used to help with simulation and debug.



Testbenches

Testbench Components

- **DUT** or Design Under Test is the Verilog module or design that you want to test. It could be a simple component like an adder or a more complex design like a microprocessor.
- The testbench itself is implemented as a separate top-level Verilog module. This module is responsible for generating input stimuli for the DUT, capturing its output, and comparing it with expected outputs.
- The testbench generates different input patterns and sequences to test different scenarios and edge cases of the design and can be coded using functions and tasks and forms the test stimulus. Some examples are the different input patterns, clock signals, reset signals, and other control signals to test various aspects of the DUT's behavior.
- Testbench signals are connected to the ports of the DUT instantiation, and are monitored by different tasks to check design functionality.

Testbenches

Testbench can be written by the following steps:

1. Declare top-level testbench module

testbench module does not need any IO ports and hence can be empty and is usually called "**tb**" or "**tb_top**", but it can be named anything.

2. Declare signals for DUT connection

The latch design contains 3 inputs and 1 output. Inputs are declared of type **reg** so that it can be driven from a procedural block such as **initial**. Outputs are declared as type **wire** so that it is visible in the testbench module and can be monitored to check design behavior.

3. Instantiate DUT

Create a module instantiation of the DUT module and connect testbench signals to the DUT ports.

Testbenches

Testbench can be written by the following steps:

4. Generate stimuli and initialize testbench variables

Use initial blocks to set starting values for your input signals, and initial/always blocks (**or a clock-generation block, if applicable**) to drive changing input patterns over time — this is what actually exercises the DUT's logic. Stimuli can be:

- **Directed**, where you manually specify each input combination or sequence you want to test, or
- **Randomized**, where inputs are generated using constructs like **\$random**, often combined with constraints to keep values meaningful.

For anything beyond a trivial design, it's good practice to wrap stimulus generation in tasks so you can reuse the same stimulus logic across multiple test scenarios without duplicating code.

5. Write checker and monitor code

This is where you verify that the DUT's actual output matches the expected output. There are two common approaches:

- **Golden model checking**: implement a simpler, trusted reference version of the design's behavior (the "golden model") in Verilog or a task/function, run it in parallel with the DUT, and compare outputs at each check point — flagging a mismatch as an error.
- **Task/function-based checkers**: write a reusable task or function, e.g. `check_output(expected)`, that compares the DUT's output against an expected value passed in, and prints a pass/fail message (often with **\$display** or **\$error**). This keeps your checking logic centralized and easy to call after every stimulus application.

Testbenches

Example

```
verilog - tb.v

// Step 1: Declare top-level testbench module
module adder_tb ();

    // Step 2: Declare signals for DUT connection
    reg clk, reset;
    reg signed [3:0] A, B;
    wire signed [4:0] C;
    integer error_count, correct_count;
    localparam MAXPOS = 7, ZERO = 0, MAXNEG = -8;

    // Step 3: Instantiate DUT
    adder DUT (.clk(clk), .reset(reset), .A(A), .B(B), .C(C));

    // Step 4: Generate stimuli (clock generation)
    initial begin
        clk = 0;
        forever #1 clk = ~clk;
    end

    // Step 4: Reusable stimulus task
    task assert_reset;
        reset = 1;
        expected_res(0);
        reset = 0;
    endtask
```

```
verilog - tb.v

// Step 4: Initialize testbench variables and generate stimuli
initial begin
    error_count = 0;
    correct_count = 0;
    A = 0; B = 0;

    assert_reset;

    A = MAXPOS; B = MAXNEG; expected_res(-1); // Edge case: opposite extremes
    A = MAXPOS; B = MAXPOS; expected_res(14); // Max positive overflow into wider result
    A = MAXNEG; B = MAXNEG; expected_res(-16); // Max negative overflow into wider result
    A = ZERO; B = ZERO; expected_res(ZERO); // Baseline case

    $display("%t: Done. Errors=%0d, Correct=%0d", $time, error_count, correct_count);
    $stop;
end

// Step 5: Write checker and monitor code
task expected_res([4:0] expected_out);
    @(negedge clk);
    if (expected_out !== C) begin
        error_count++;
        $display("%t: Error: A=%0d, B=%0d -> expected C=%0d, got C=%0d",
            $time, A, B, expected_out, C);
    end else begin
        correct_count++;
    end
endtask

endmodule
```

Testbenches

Testbench Simulation

Simulation is a technique of applying different input stimulus to the design at different times to check if the RTL code behaves the intended way. Essentially, simulation is a well-followed technique to verify the robustness of the design. It is also similar to how a fabricated chip will be used in the real world and how it reacts to different inputs.

```
verilog - tb.v

// Step 1: Declare top-level testbench module
module tb_sub_12bit_pipeline;

    // Step 2: Declare signals for DUT connection
    reg clk, rst_n;
    reg signed [11:0] a, b;
    wire signed [11:0] result;
    wire valid;

    // Step 3: Instantiate DUT
    sub_12bit_pipeline dut (.clk(clk), .rst_n(rst_n), .a(a), .b(b), .result(result), .valid(valid));

    // Step 4: Generate stimuli (clock generation)
    initial begin clk = 0; forever #1 clk = ~clk; end

    // Step 4: Initialize variables and generate stimuli
    initial begin
        rst_n = 0; a = 0; b = 0;
        #20 rst_n = 1;
        @(negedge clk) rst_n = 0;
        repeat(2) @(negedge clk);

        a = 12'sd100;   b = 12'sd50;   #10; // Test 1: 100 - 50 = 50
        a = -12'sd100; b = 12'sd50;   #10; // Test 2: -100 - 50 = -150
        a = 12'sd0;    b = -12'sd2048; #10; // Test 3: 0 - (-2048), overflow edge
        a = -12'sd2048; b = 12'sd2047; #10; // Test 4: min - max, wraps

        #30 $stop; // wait for pipeline latency (2 cycles), then stop
    end

    // NOTE: Step 5 can be skipped entirely if you just want to visually inspect
    // the design's behavior – dump signals to a waveform and see the results instead of writing an automated checker.

endmodule
```

References

References

- [ChipVerify - Verilog HDL Tutorials & Examples](#)
- [VLSI Verify - Verilog Tutorials & Design Guide](#)
- [YouTube - Verilog HDL Video Tutorial](#)

FRIENDLY REMINDER

Israel has killed more than 13,000 children in Gaza since October 7 while others are suffering from severe malnutrition and do not “even have the energy to cry”, says the United Nations Children’s Fund (UNICEF).

“Thousands more have been injured or we can’t even determine where they are. They may be stuck under rubble ... We haven’t seen that rate of death among children in almost any other conflict in the world”

-UNICEF Executive Director

Save The Palestinian Children

THANKS
